Computer Networks (CS-241)

Aqsa Safdar
Department of Computer Science,
University of Wah

Adapted From:
Computer Networking: A Top-Down Approach
8th edition: Jim Kurose, Keith Ross Pearson, 2020

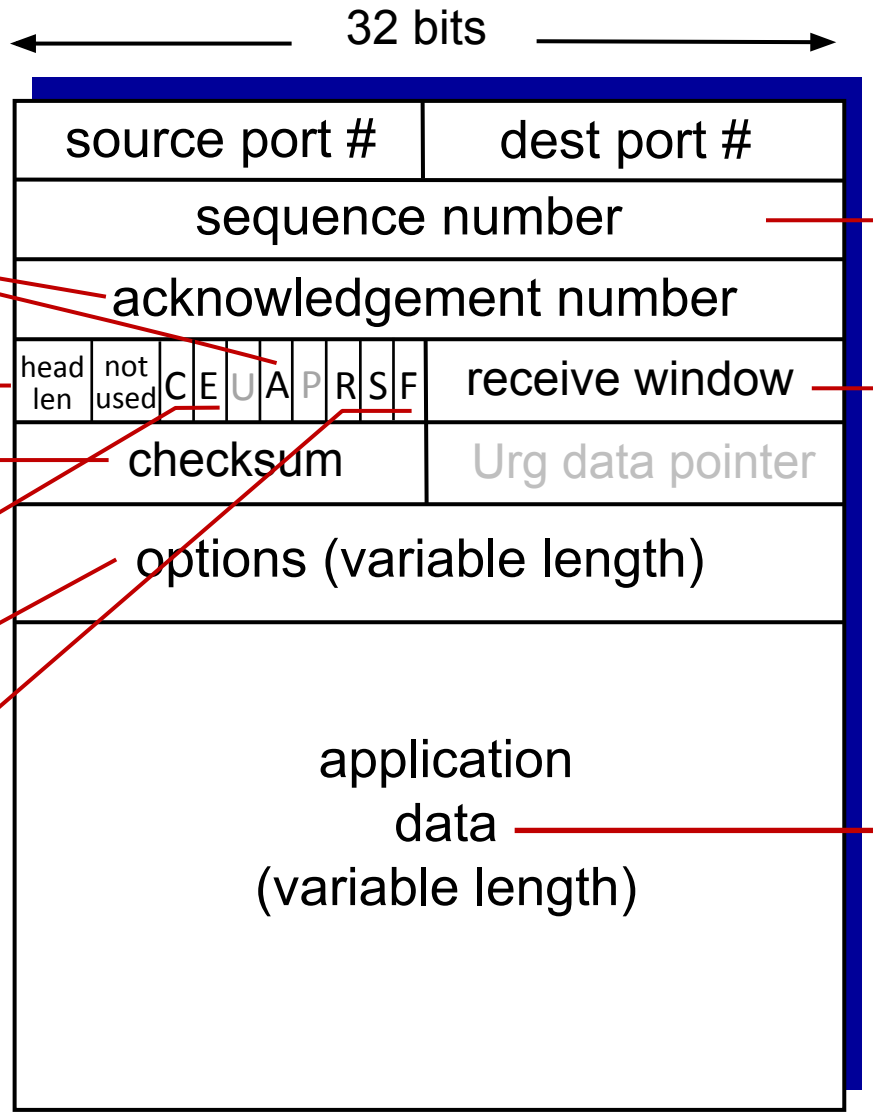
Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
  - segment structure
  - reliable data transfer
  - flow control
  - connection management
- Principles of congestion control
- TCP congestion control

TCP: overview RFCs: 793,1122, 2018, 5681, 7323

- point-to-point:
  - one sender, one receiver
- reliable, in-order *byte steam:*
  - no “message boundaries"
- full duplex data:
  - bi-directional data flow in same connection
  - MSS: maximum segment size
- cumulative ACKs
- pipelining:
  - TCP congestion and flow control set window size
- connection-oriented:
  - handshaking (exchange of control messages) initializes sender, receiver state before data exchange
- flow controlled:
  - sender will not overwhelm receiver

TCP segment structure



ACK: seq # of next expected byte; A bit: this is an ACK

length (of TCP header)

Internet checksum

C, E: congestion notification

TCP options

RST, SYN, FIN: connection management

segment seq #: counting bytes of data into bytestream (not segments!)

flow control: # bytes receiver willing to accept

data sent by application into TCP socket

Chapter 3: roadmap

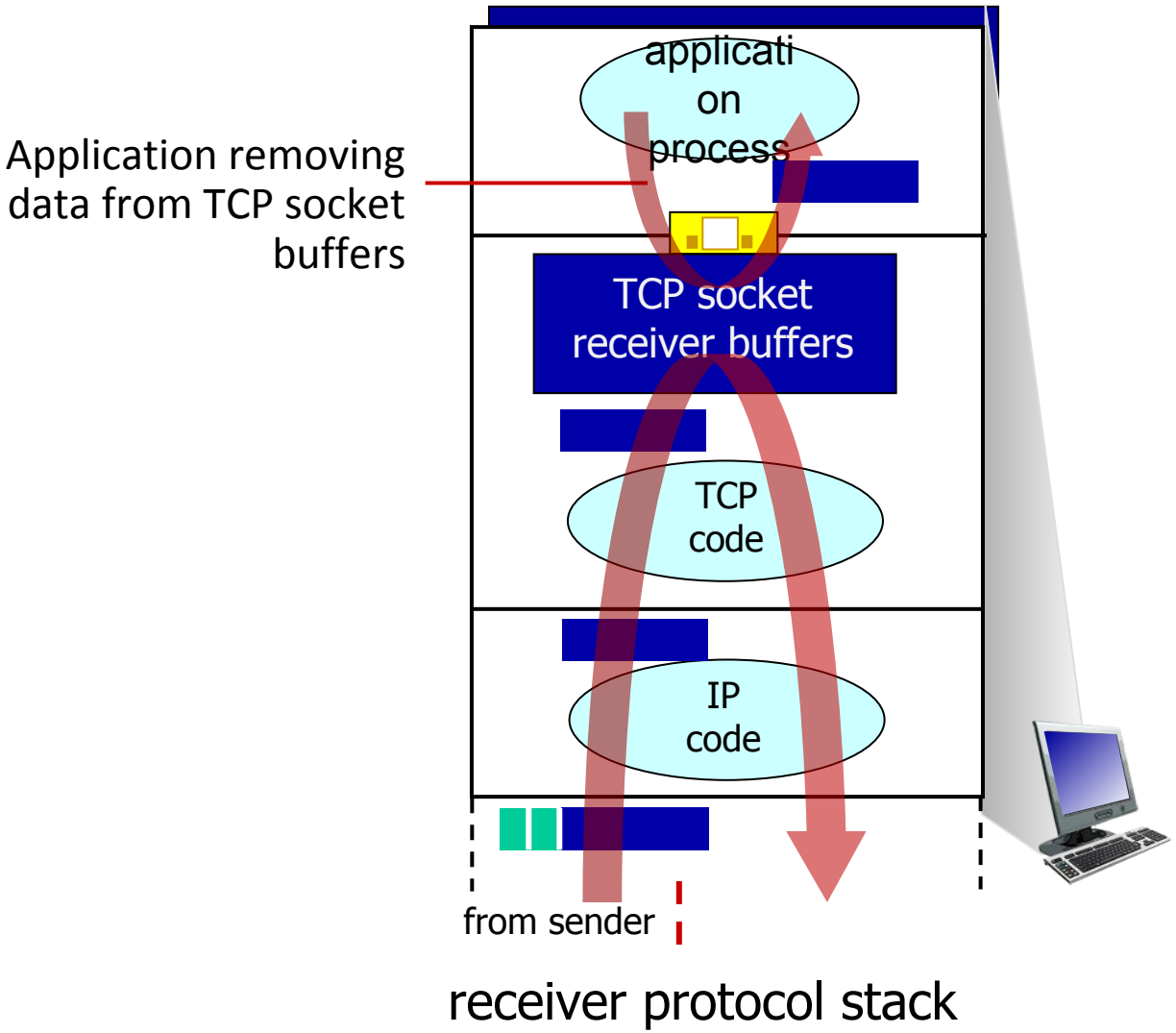
- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
  - segment structure
  - reliable data transfer
  - flow control
  - connection management
- Principles of congestion control
- TCP congestion control

TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?

flow control

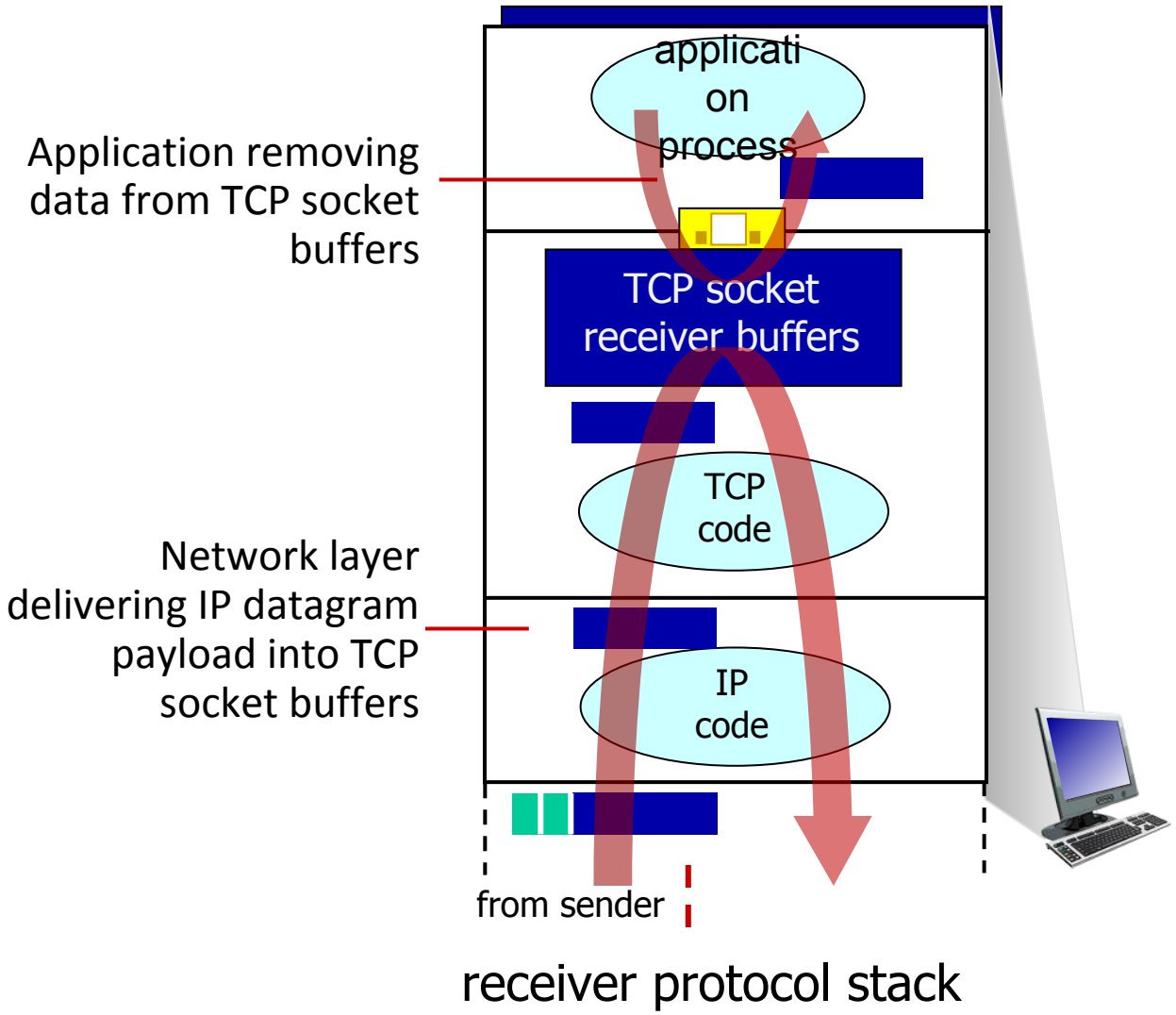
receiver controls sender, so sender won't overflow receiver's buffer by transmitting too much, too fast



TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?

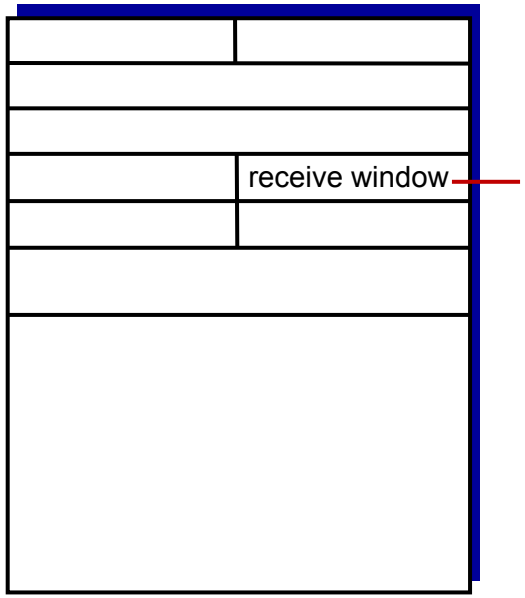
A: If the network layer delivers data faster than the application removes (read) it from the socket buffer, the receive buffer fills up, TCP will not discard the data (normally),so TCP advertises a smaller window, and eventually a **zero window.** This causes the sender to stop sending until the application reads more data. This mechanism is TCP flow control.



receiver protocol stack

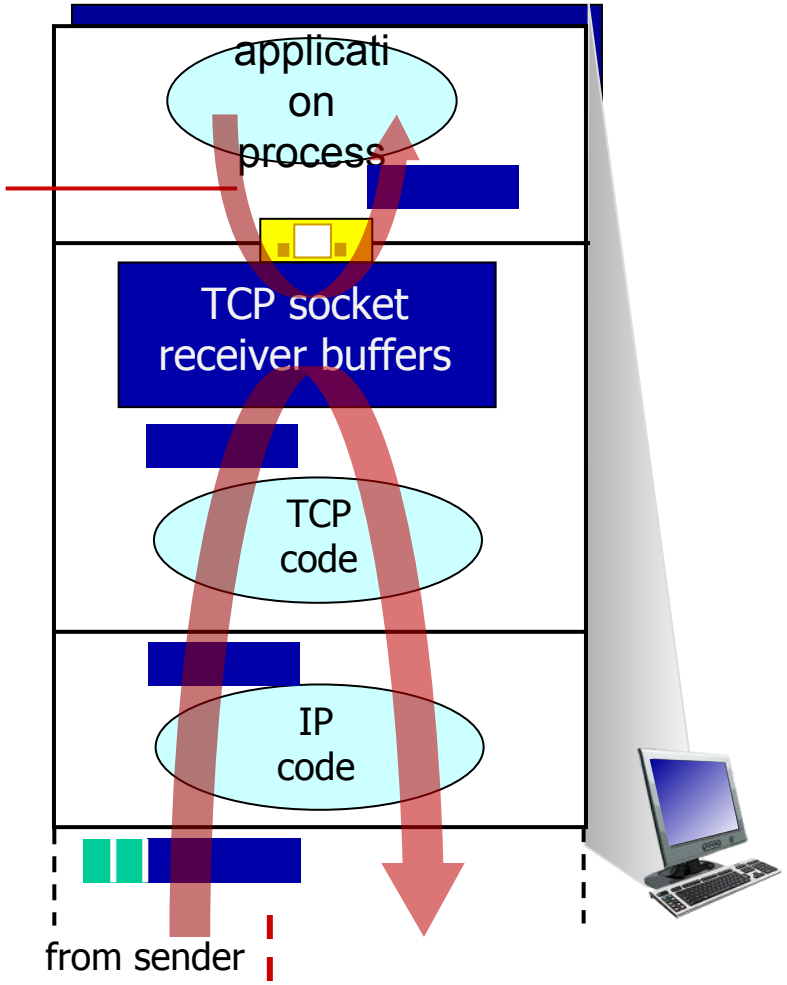
TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?



flow control: # bytes receiver willing to accept

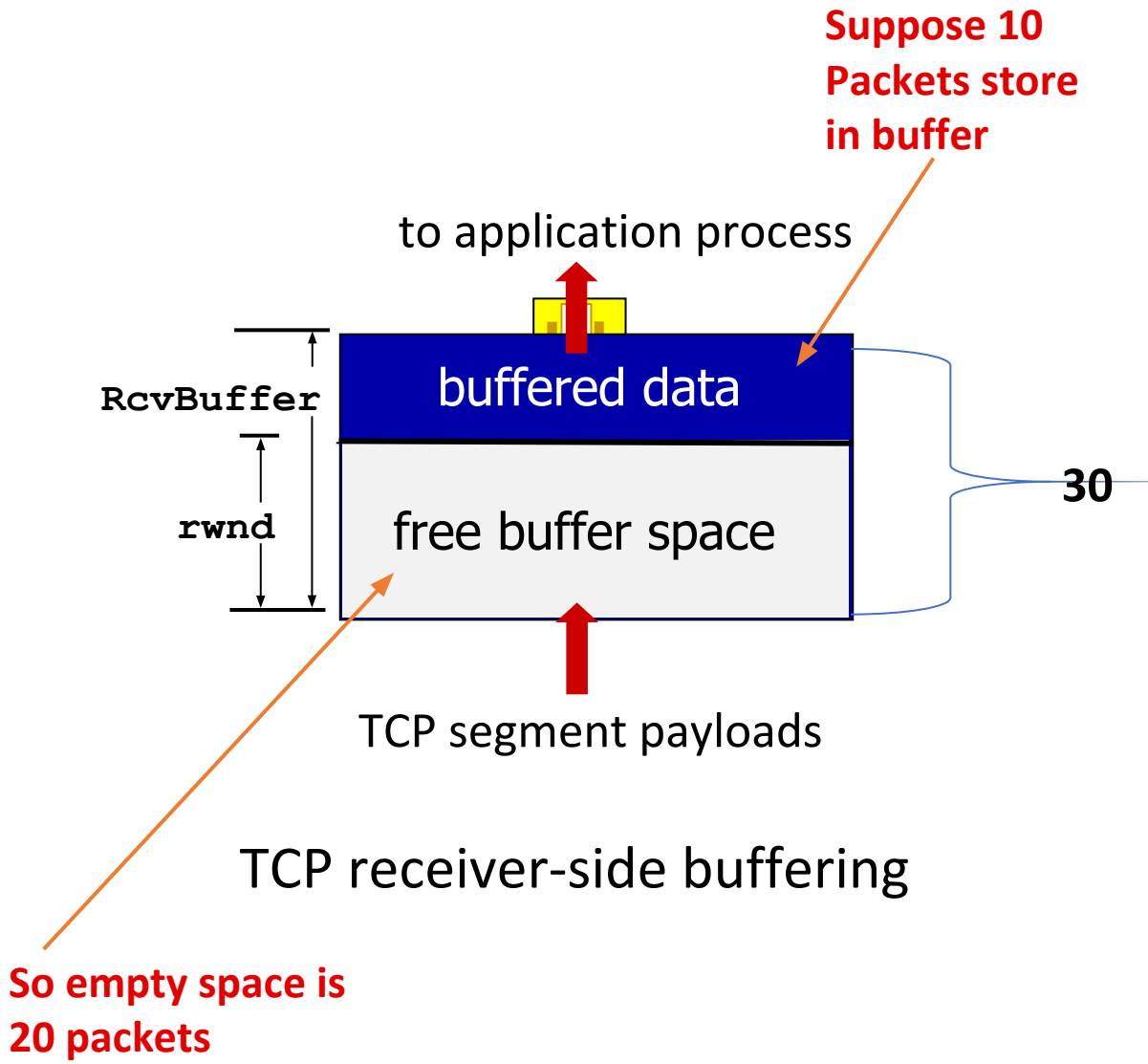
Application removing data from TCP socket buffers



receiver protocol stack

TCP flow control

- TCP receiver "advertises" free buffer space in **`rwnd`** (receive window) field in TCP header. This tells the sender how much more data it is allowed to send.
  - **`RcvBuffer`** size set via socket options (typical default is 4096 bytes)
  - many operating systems auto adjust **`RcvBuffer`**
- Sender limits amount of unACKed ('in-flight' or in buffer) data to received rwnd.
- guarantees receive buffer will not overflow

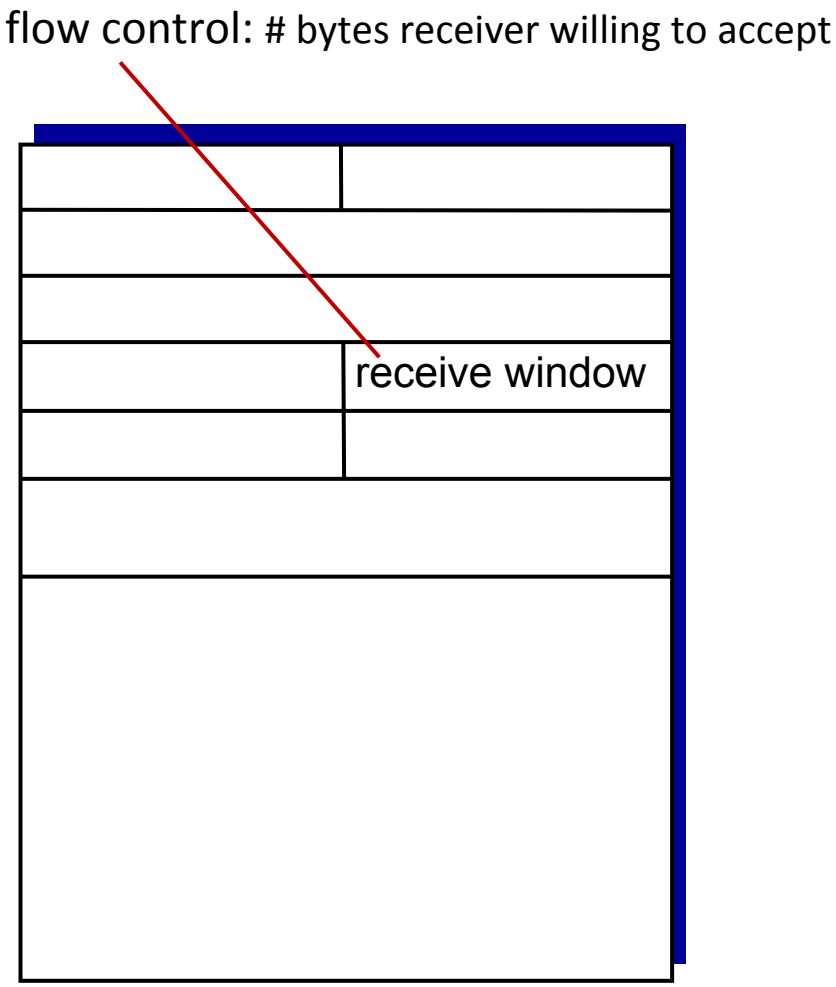


TCP receiver-side buffering

TCP flow control

- TCP receiver "advertises" free buffer space in **`rwnd`** field in TCP header
  - **`RcvBuffer`** size set via socket options (typical default is 4096 bytes)
  - many operating systems autoadjust **`RcvBuffer`**
- sender limits amount of unACKed ("in-flight") data to received **`rwnd`**
- guarantees receive buffer will not overflow

LastByteSent-LastByteAcked<=rwnd



TCP segment format

Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- **Principles of congestion control**
- TCP congestion control
- Evolution of transport-layer functionality

Principles of congestion control

Congestion:

- informally: “too many sources sending too much data too fast for *network* to handle”
- manifestations:
  - long delays (queueing in router buffers)
  - packet loss (buffer overflow at routers)
- different from flow control!
- a top-10 problem!

congestion control: too many senders, sending too fast

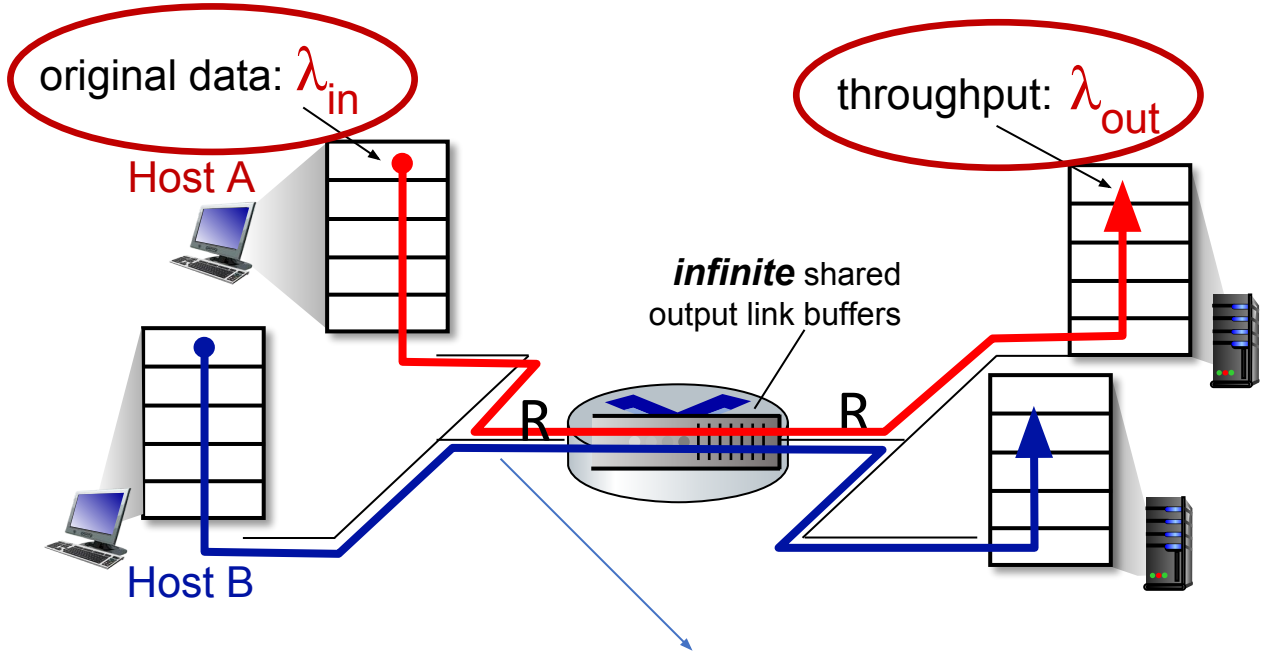
flow control: one sender too fast for one receiver

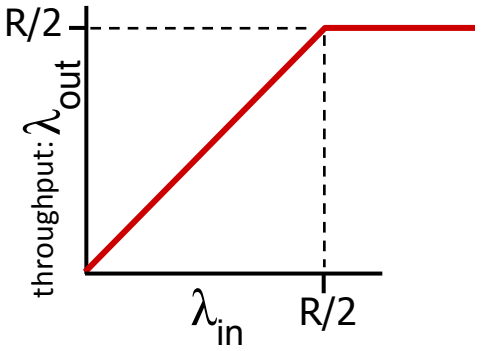
Causes/costs of congestion: scenario 1

Simplest scenario:

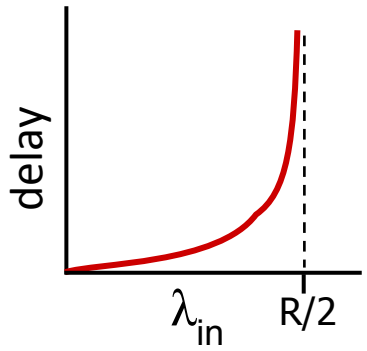
- Two senders,two receivers
  - one router, infinite buffers
  - input, output link capacity: R
  - two flows
  - no retransmissions needed
  - Congestion only affects delay, not packet loss (since buffers are infinite).

Q: What happens as arrival rate λ_{in} approaches R/2?





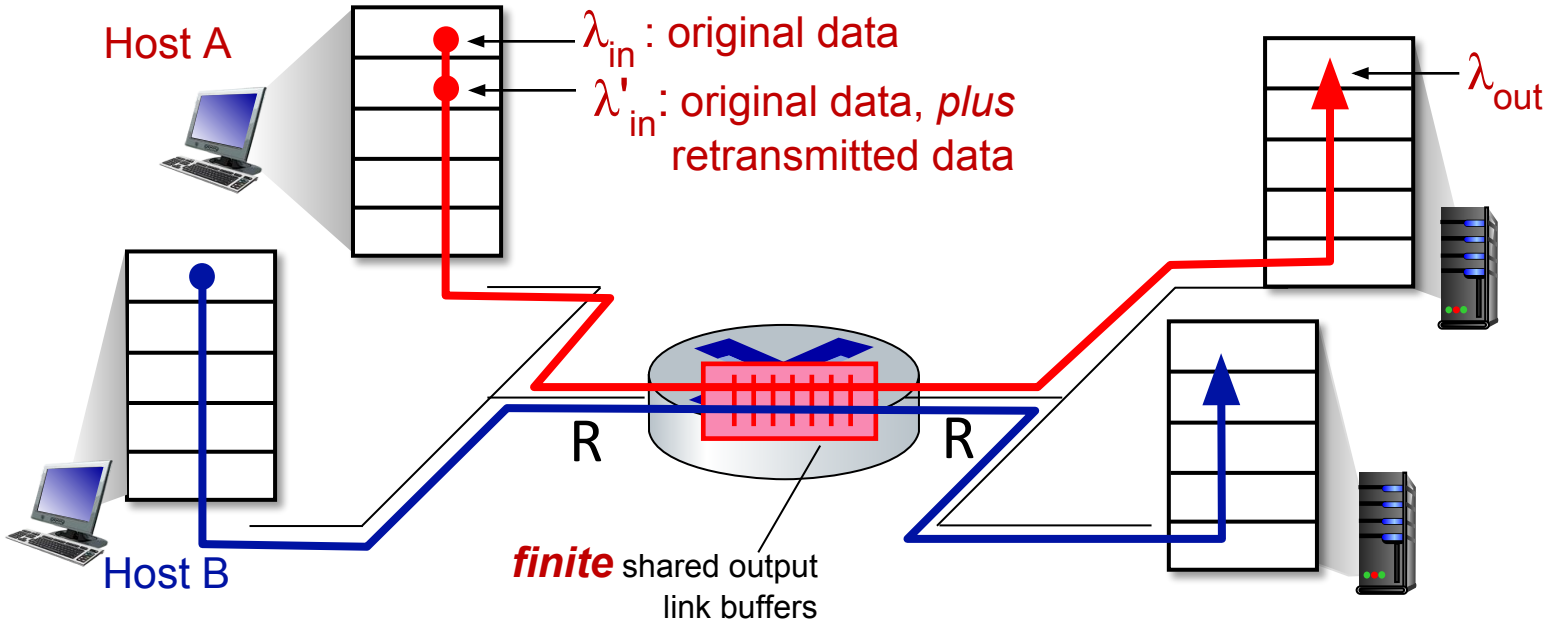
maximum per-connection throughput: R/2



large delays as arrival rate λ_{in} approaches capacity

Causes/costs of congestion: scenario 2

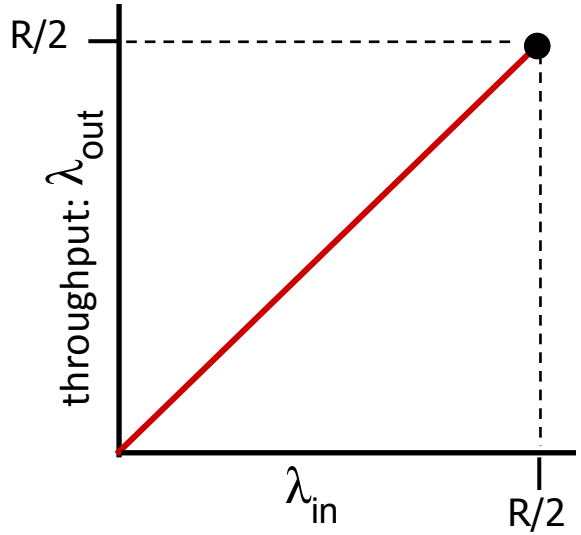
- one router, *finite* buffers so packet will loss
- When the **router's buffer** is full: New packets cannot be stored. They are dropped. The sender times out. The sender retransmits the dropped packet.
- The application sends original data at some rate (λ_in)and The receiver gets the same original data (λ_out). So for the application: λ_in = λ_out
- Because some packets are dropped, TCP retransmits them. So TCP's actual input becomes: original data + retransmitted data **λ'_in ≥ λ_in**

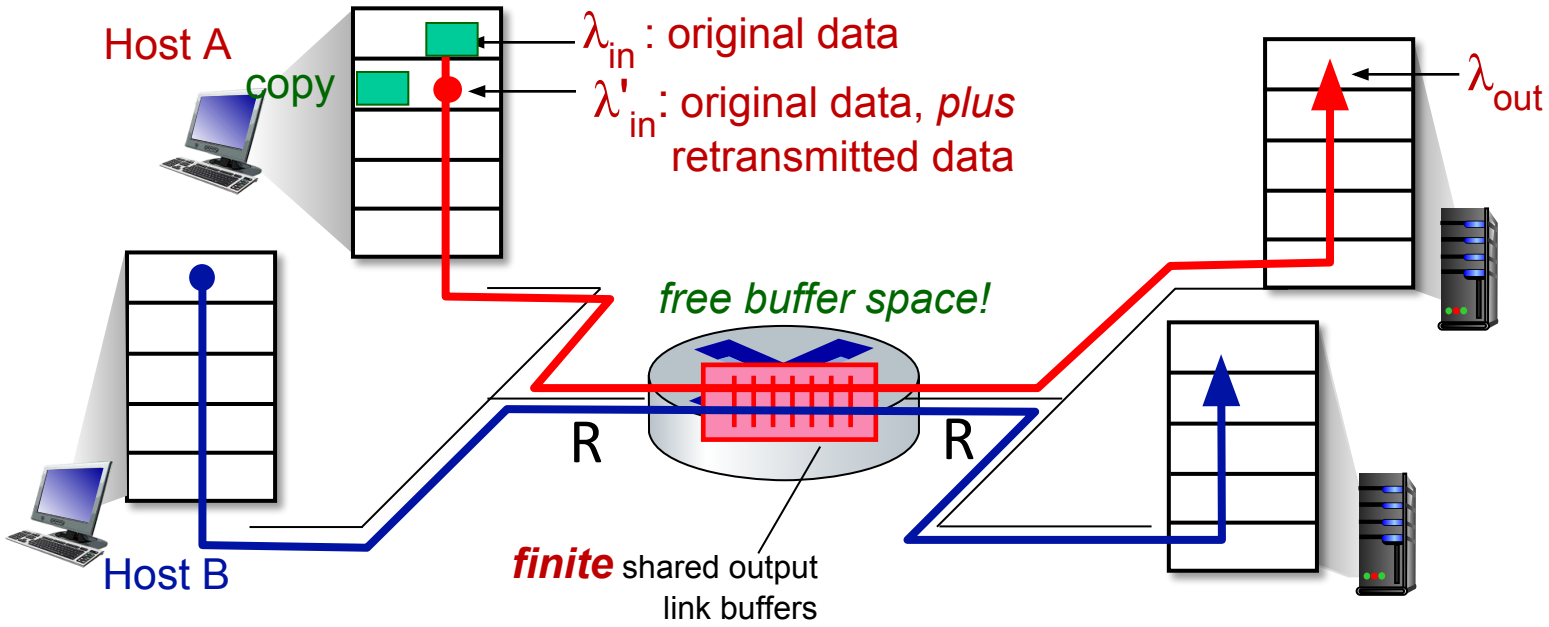


Causes/costs of congestion: scenario 2

Idealization: perfect knowledge

- sender sends only when router buffers available
- This means: No packet loss
- No retransmissions
- No congestion collapse

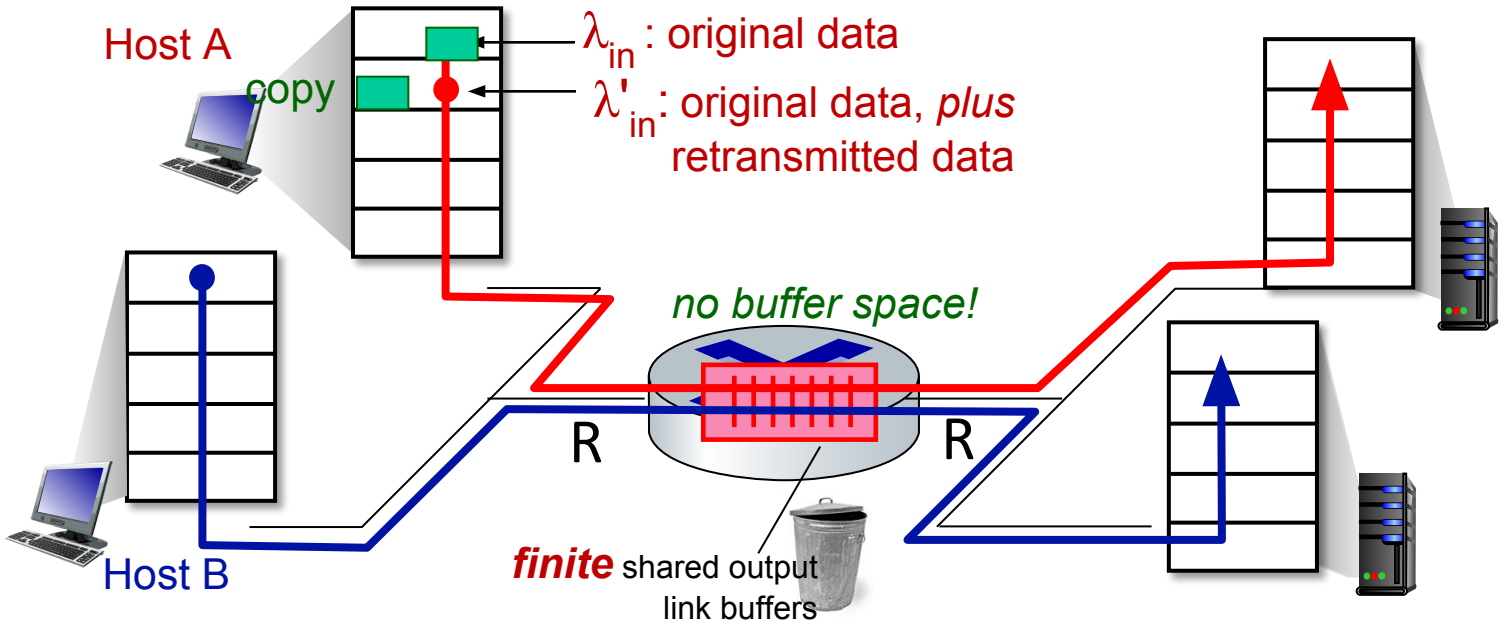




Causes/costs of congestion: scenario 2

Idealization: *some* perfect knowledge

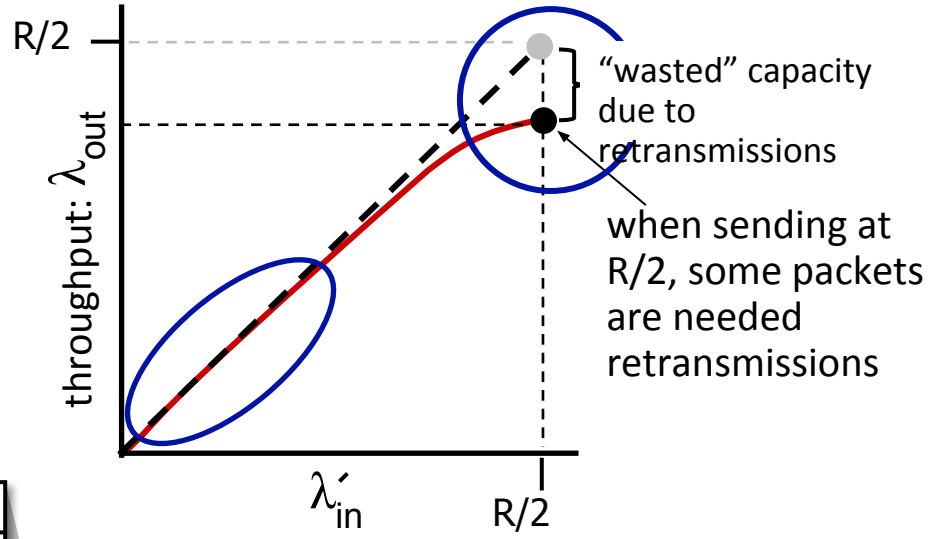
- packets can be lost (dropped at router) due to full buffers
- sender knows when packet has been dropped: It only retransmits when it is 100% sure the packet is lost.

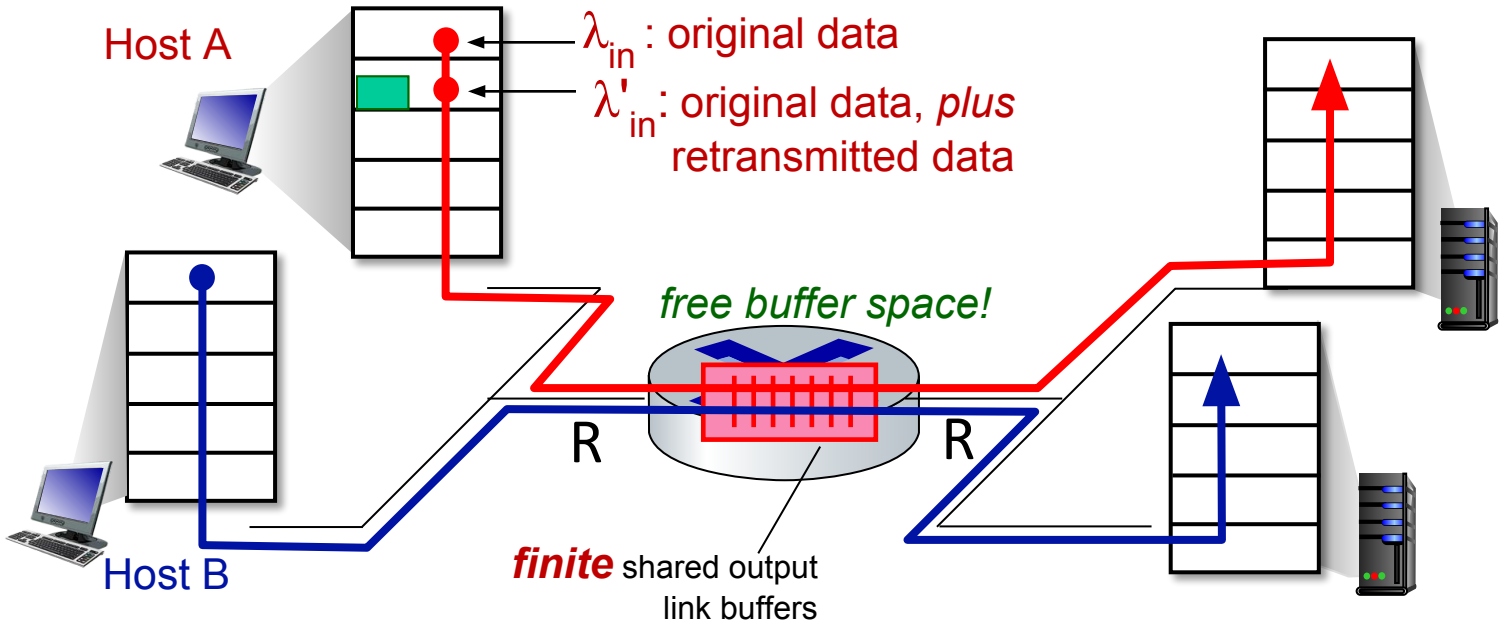


Causes/costs of congestion: scenario 2

Idealization: *some* perfect knowledge

- packets can be lost (dropped at router) due to full buffers
- sender knows when packet has been dropped: only resends if packet *known* to be lost

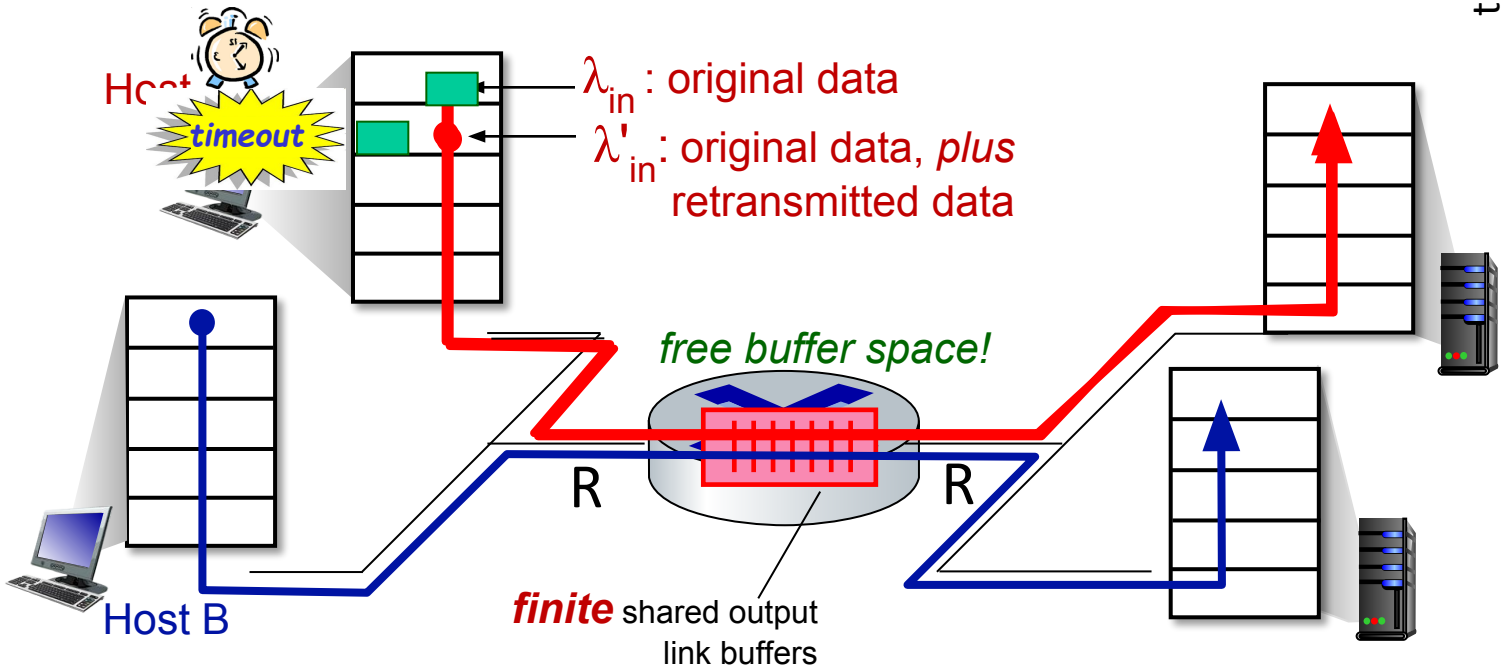


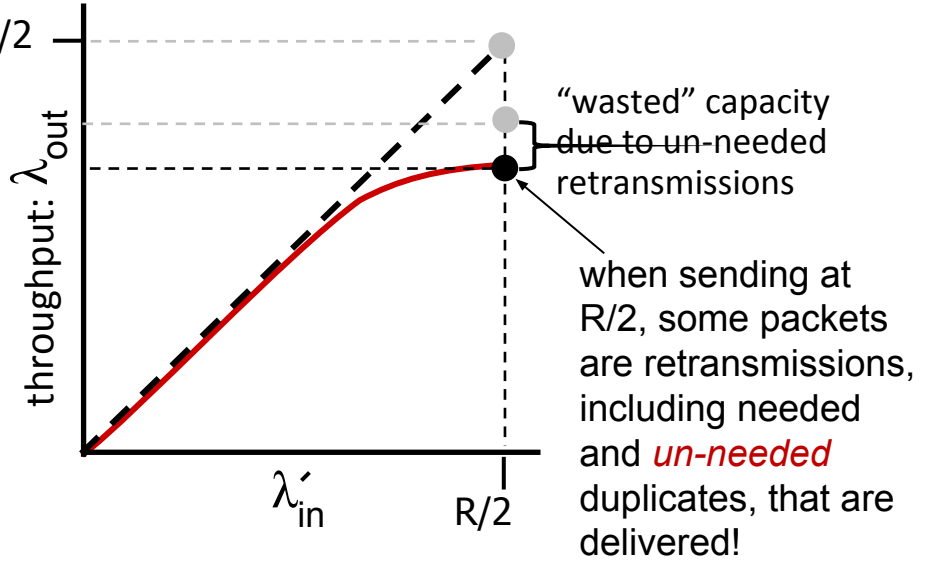


Causes/costs of congestion: scenario 2

Realistic scenario: *un-needed duplicates*

- packets can be lost, dropped at router due to full buffers – requiring retransmissions
- but sender times can time out prematurely, means Original packet late not lost but sender send *two* copies, *both* of which are delivered which waste network capacity.

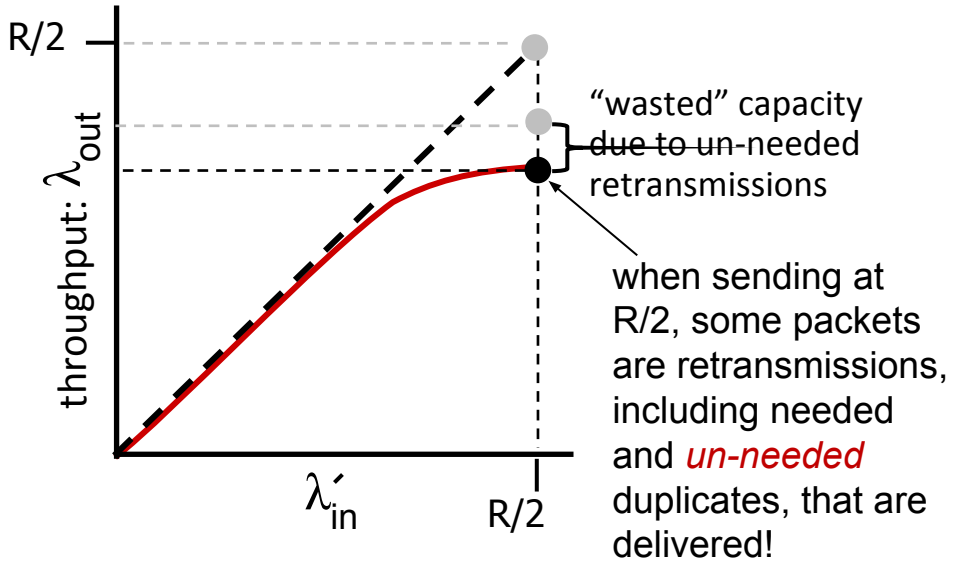




Causes/costs of congestion: scenario 2

Realistic scenario: *un-needed duplicates*

- packets can be lost, dropped at router due to full buffers – requiring retransmissions
- but sender times can time out prematurely, sending *two* copies **(original + retransmitted),** *both* of which are delivered



"costs" of congestion:

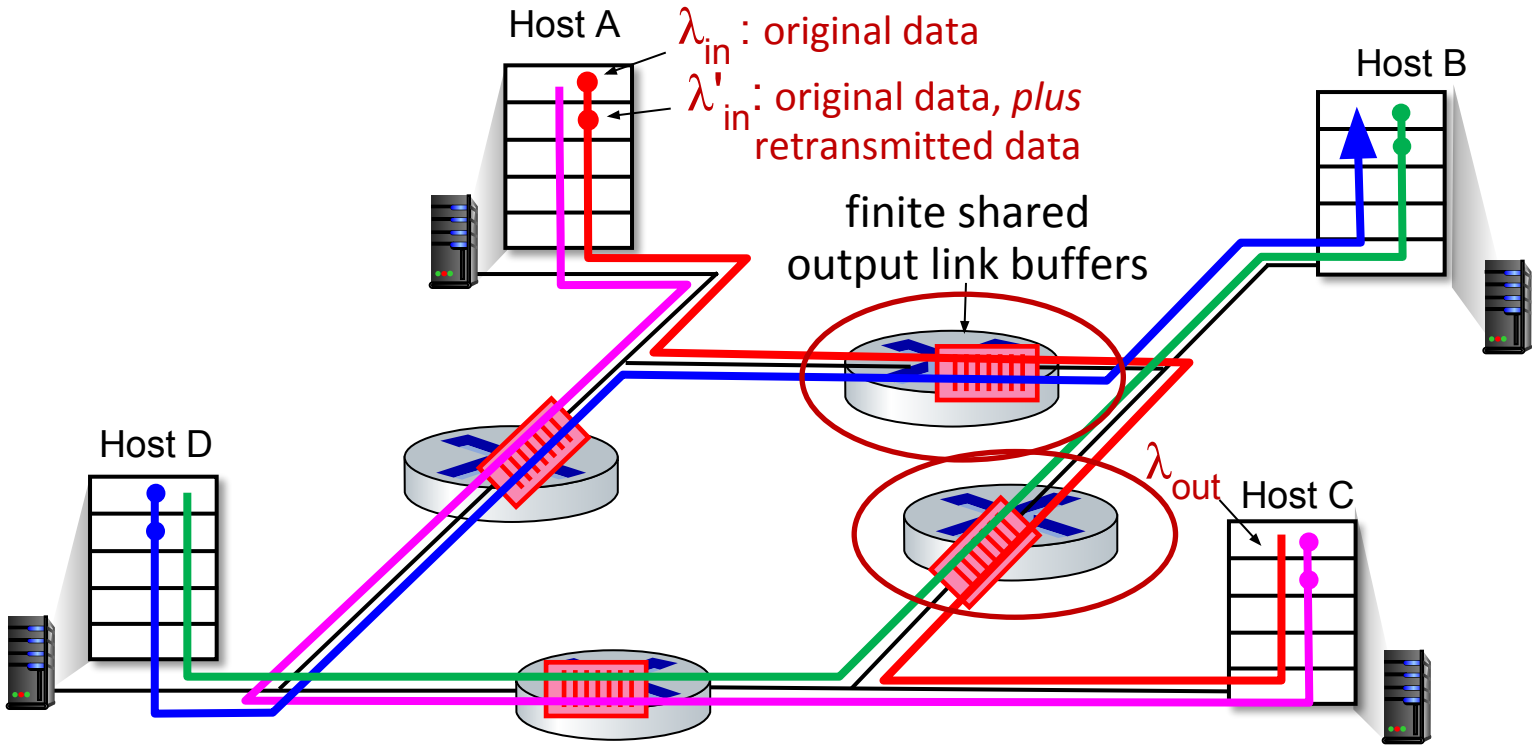
- When congestion causes packet loss, TCP must retransmit them, increasing work for the sender and network to deliver the same data.
- **unneeded retransmissions:** link carries multiple copies of a packet
  - So congestion reduces the maximum possible data rate the network can achieve.

Causes/costs of congestion: scenario 3

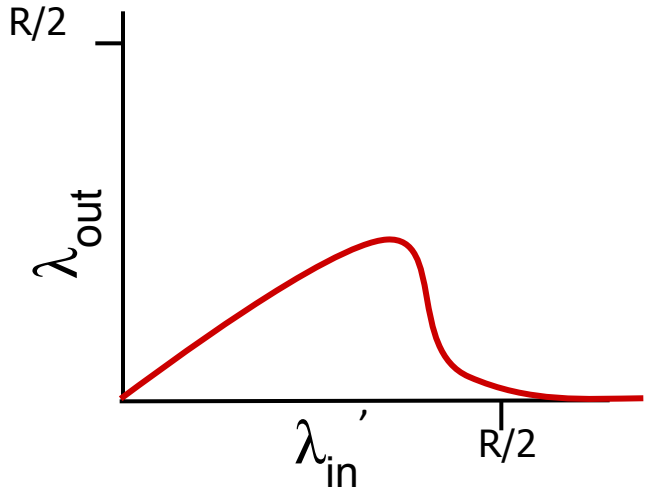
- *four* senders
- *multi-hop* paths
- timeout/retransmit

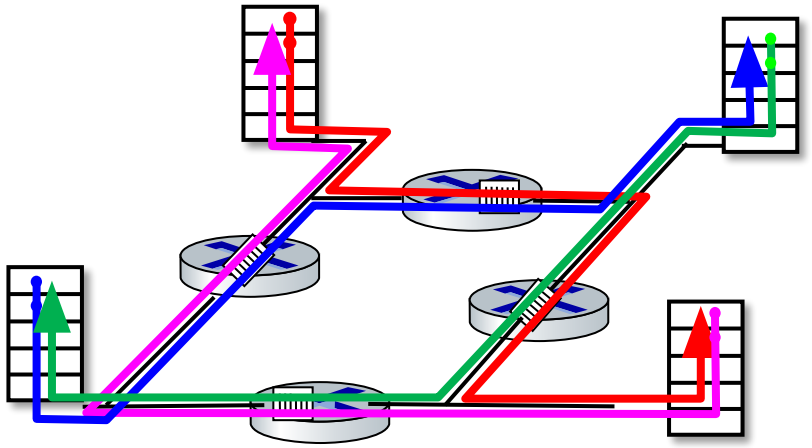
Q: what happens as λ_{in} and λ_{in}' increase ?

A: as red λ_{in}' increases, all arriving blue pkts at upper queue are dropped, blue throughput → 0



Causes/costs of congestion: scenario 3



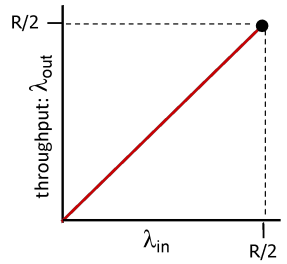


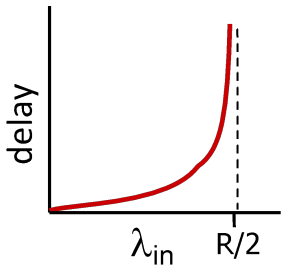
another "cost" of congestion:

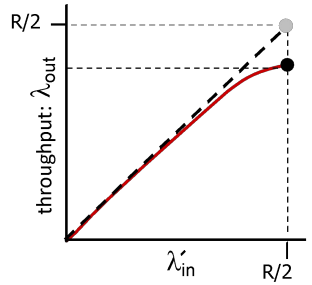
- when packet dropped, any upstream transmission capacity and buffering used for that packet was wasted!

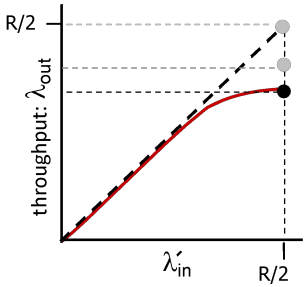
Causes/costs of congestion: insights

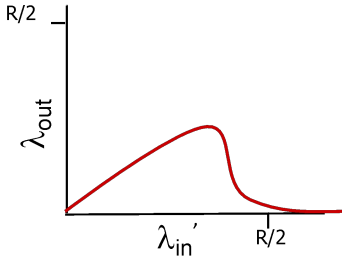
- throughput can never exceed capacity
- Throughput ≤ Capacity
- delay increases as capacity approached
- loss/retransmission decreases effective throughput
- un-needed duplicates further decreases effective throughput
- upstream transmission capacity / buffering wasted for packets lost downstream









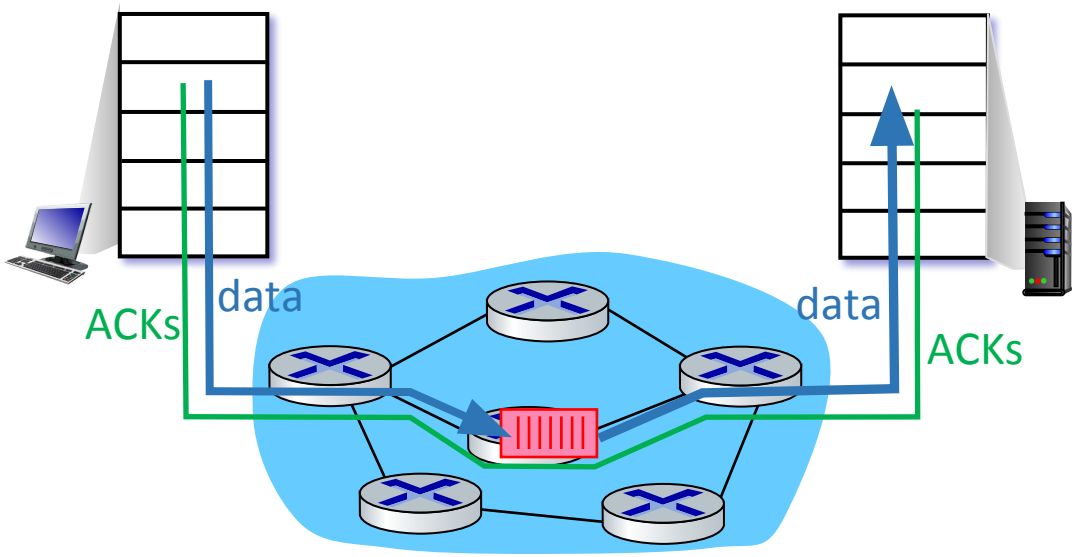


Link A → B → C → D ← Congestion here so A to **C** every thing wasted

Approaches towards congestion control

End-end congestion control:

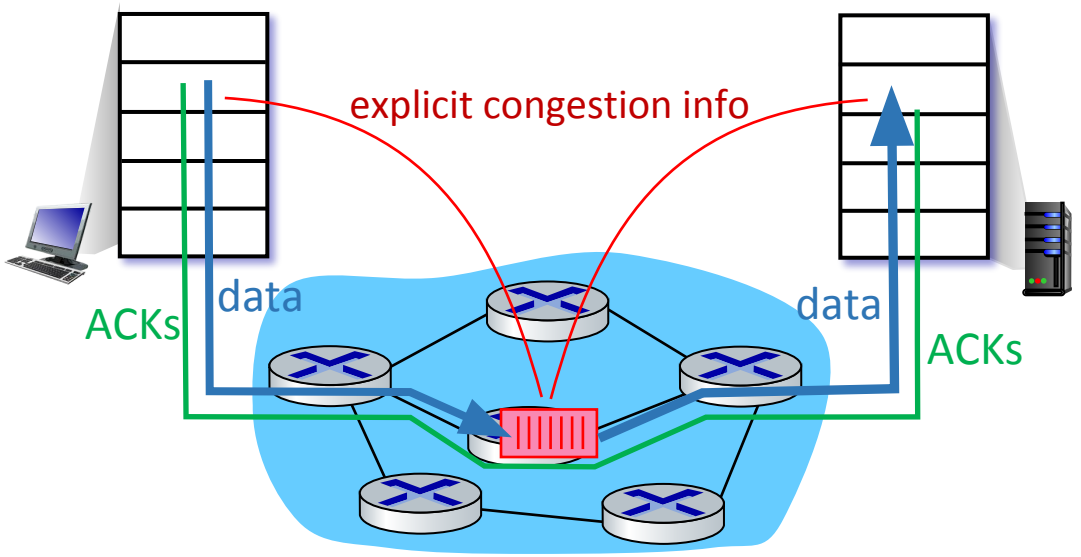
- no explicit feedback from network
- The sender observes packet loss (timeouts, missing ACKs) and increased delay (ACKs take longer).These signals suggest that the network is congested.
- TCP interprets packet loss as a sign of congestion.



Approaches towards congestion control

Network-assisted congestion control:

- Routers monitor congestion in real time (queue lengths, buffer occupancy).
- When congestion is detected, they notify the sender or receiver.
- Explicit congestion signals (ECN, DECbit): Router marks packets to indicate congestion.
- TCP ECN, ATM, DECbit protocols



Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality

TCP congestion control: AIMD

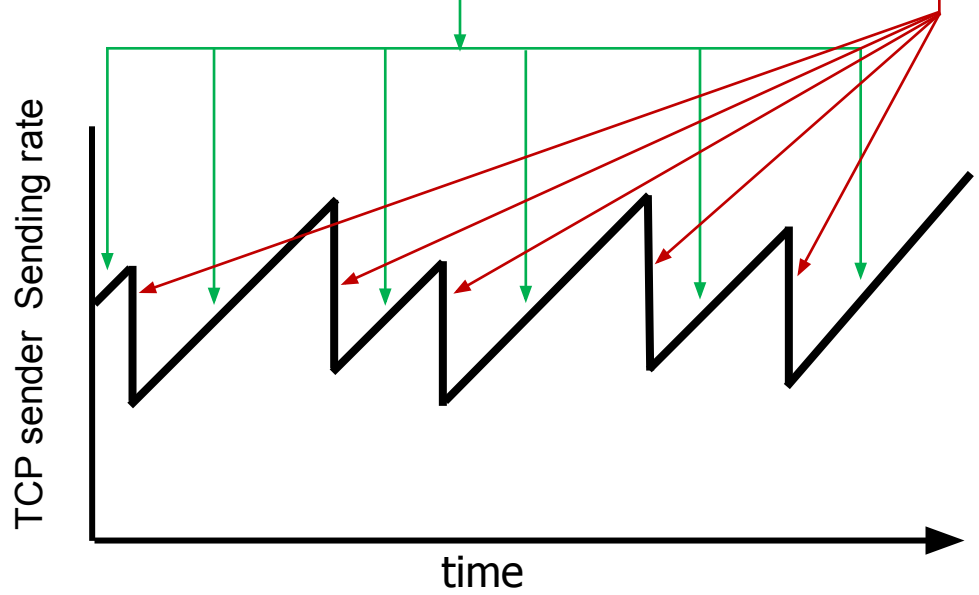
- *approach:* senders can increase sending rate until packet loss (congestion) occurs, then decrease sending rate on loss event

Additive Increase

increase sending rate by 1 maximum segment size every RTT until loss detected

Multiplicative Decrease

cut sending rate in half at each loss event



AIMD This is how TCP controls congestion in the network

TCP AIMD: more

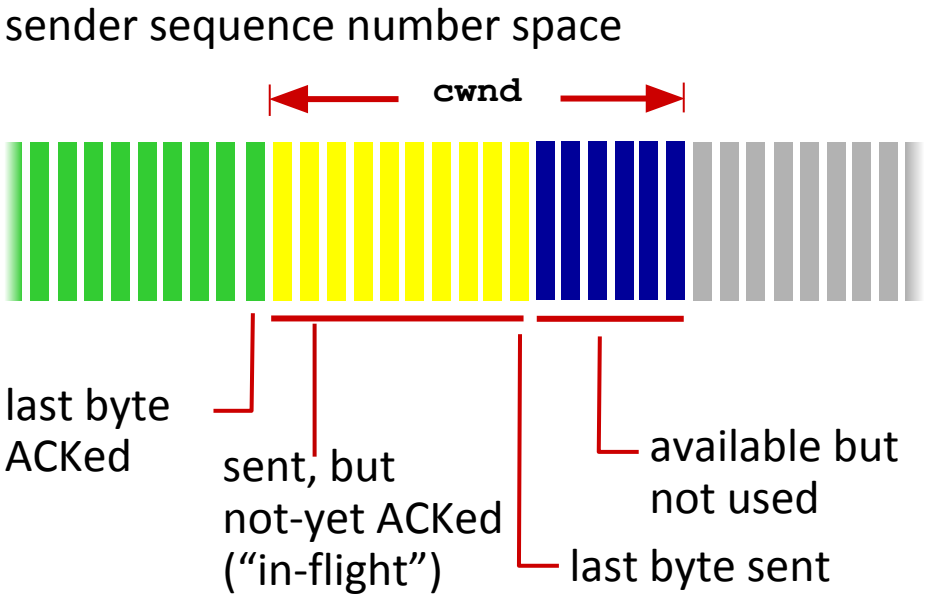
Multiplicative decrease detail: sending rate is

- Cut in half on loss detected by triple duplicate ACK (TCP Reno)
- Cut to 1 MSS (maximum segment size) when loss detected by timeout (TCP Tahoe)

Why AIMD?

- AIMD – a distributed, asynchronous algorithm – has been shown to:
  - achieves optimal throughput
  - ensures fair bandwidth sharing
  - provides stable behavior

TCP congestion control: details



TCP sending behavior:

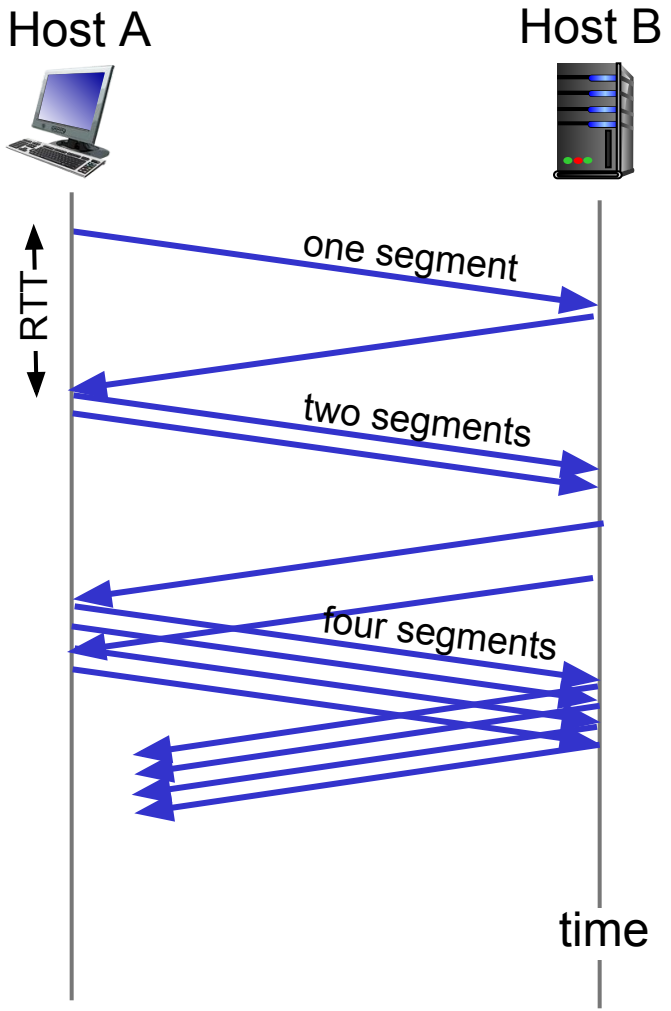
- *roughly:* send `cwnd` bytes, wait RTT for ACKS, then send more bytes

$$\text{TCP rate} \approx \frac{\texttt{cwnd}}{\text{RTT}} \text{ bytes/sec}$$

- TCP sender limits transmission: `LastByteSent- LastByteAcked ≤ cwnd`
- `cwnd` is dynamically adjusted in response to observed network congestion (implementing TCP congestion control)

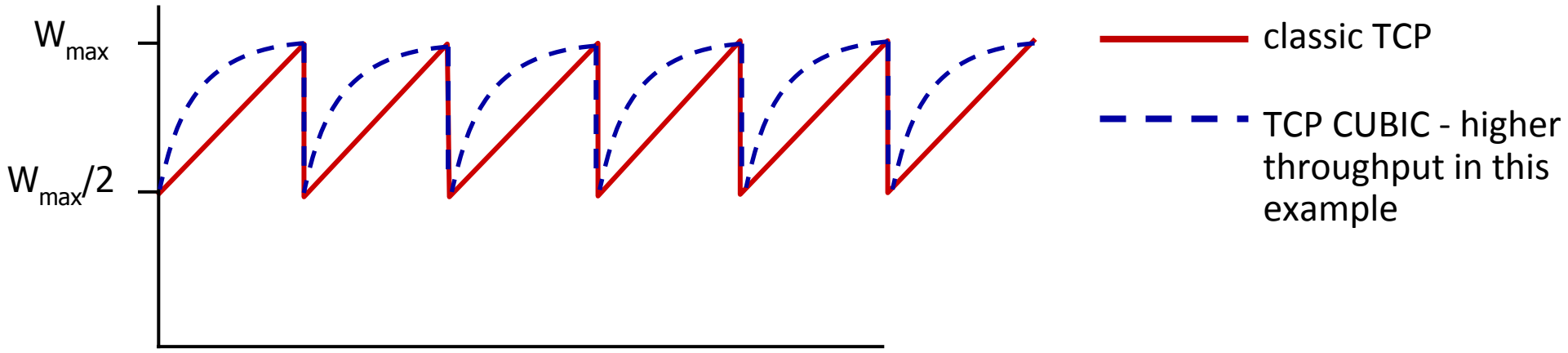
TCP slow start

- when connection begins, increase rate exponentially until first loss event:
  - initially **`cwnd`** = 1 MSS
  - double **`cwnd`** every RTT
  - done by incrementing **`cwnd`** for every ACK received
- *summary:* initial rate is slow, but ramps up exponentially fast



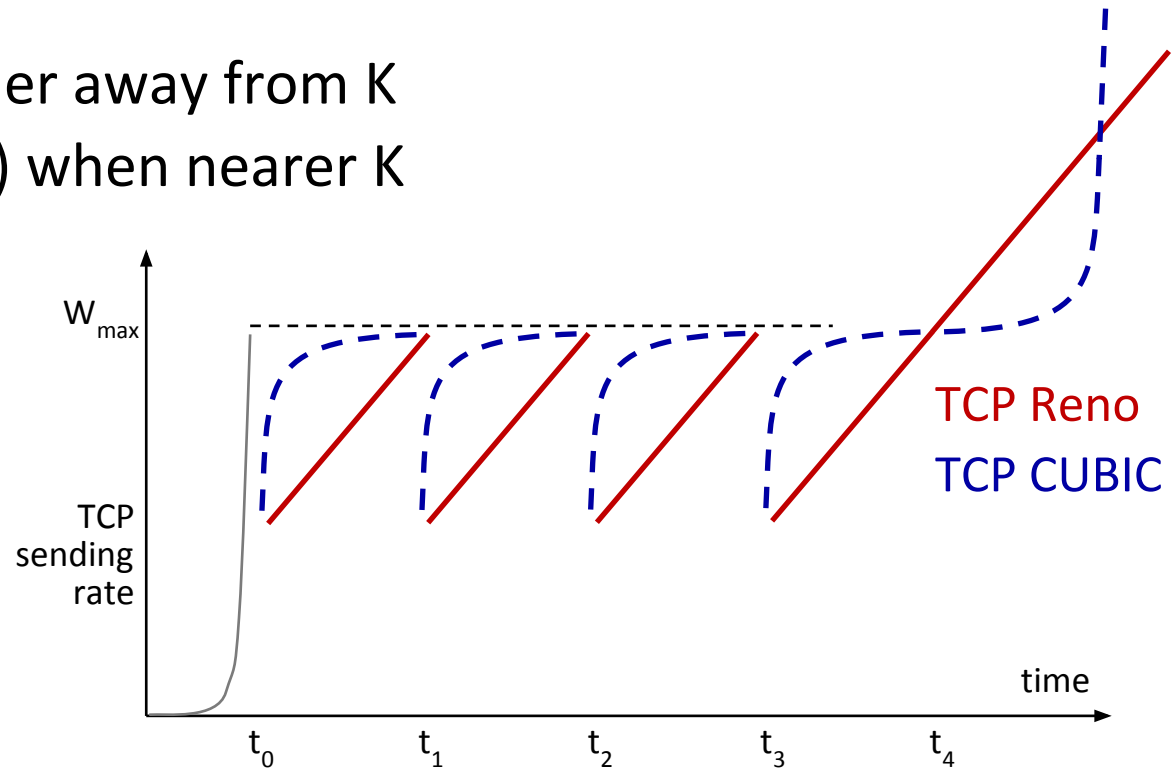
TCP CUBIC

- Is there a better way than AIMD to "probe" for usable bandwidth?
- Insight:
  - W_{max}: sending rate at which congestion loss was detected
  - congestion state of bottleneck link probably (?) hasn't changed much
  - after cutting rate/window in half on loss, initially ramp to to W_{max} *faster*, but then approach W_{max} more *slowly*



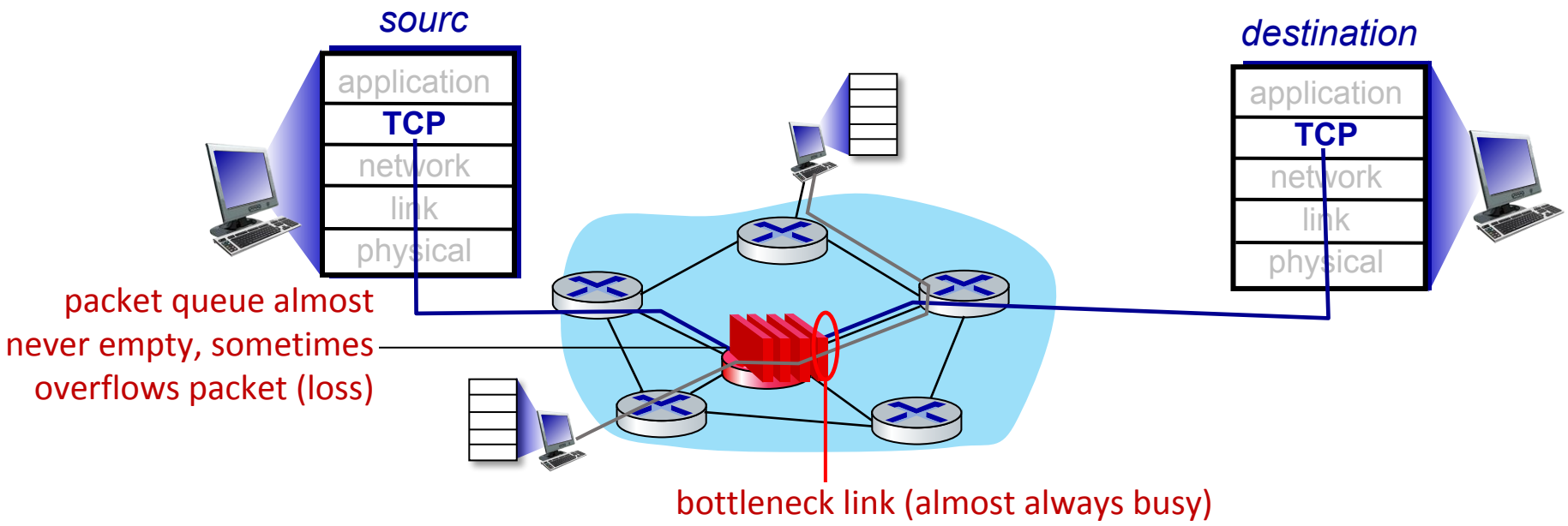
TCP CUBIC

- K: point in time when TCP window size will reach W_{max}
  - K itself is tuneable
- increase W as a function of the *cube* of the distance between current time and K
  - larger increases when further away from K
  - smaller increases (cautious) when nearer K
- TCP CUBIC default in Linux, most popular TCP for popular Web servers



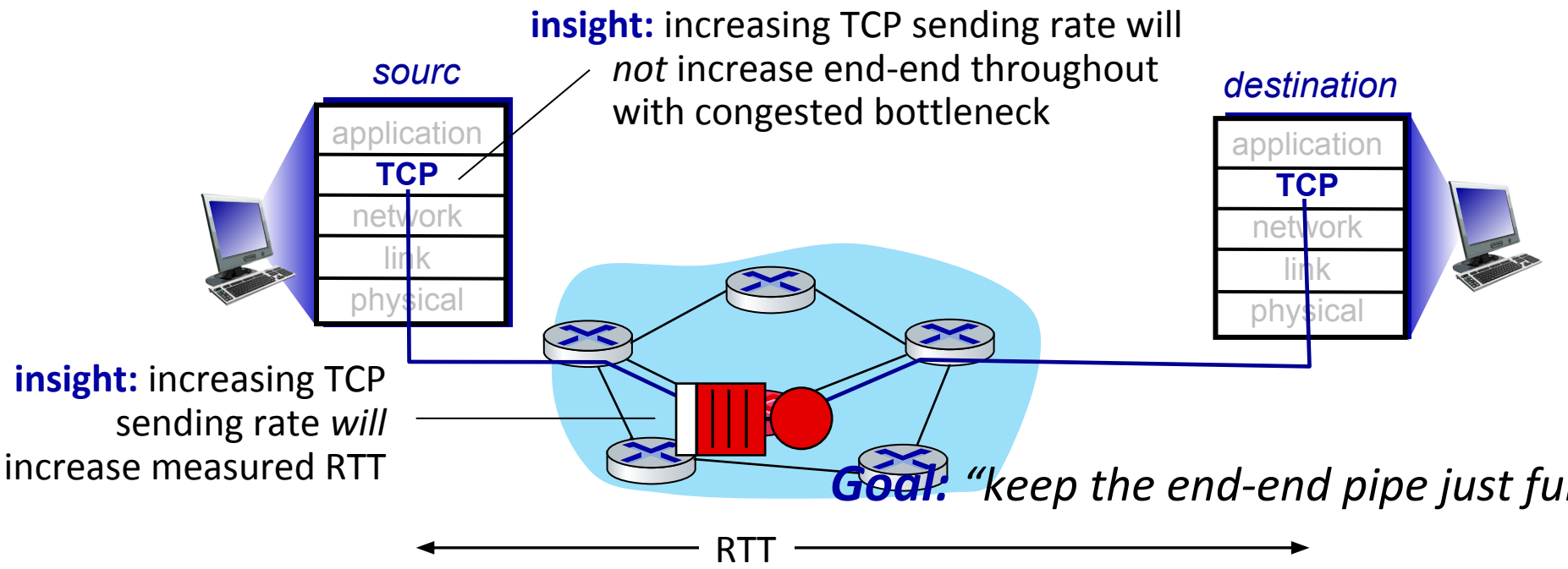
TCP and the congested "bottleneck link"

- TCP (classic, CUBIC) increase TCP's sending rate until packet loss occurs at some router's output: the *bottleneck link*



TCP and the congested "bottleneck link"

- TCP (classic, CUBIC) increase TCP's sending rate until packet loss occurs at some router's output: the *bottleneck link*
- understanding congestion: useful to focus on congested bottleneck link

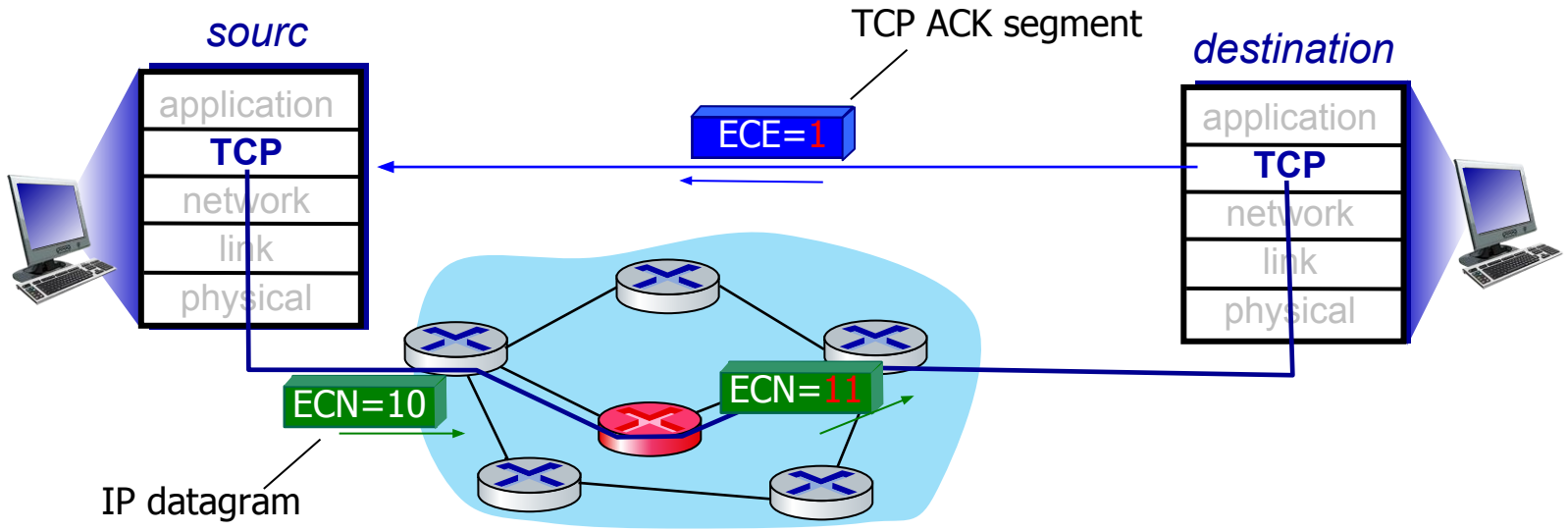


Delay-based TCP congestion control

- Traditional TCP (Reno, CUBIC) often detects congestion only after packet loss. But **loss = congestion already happened** → not ideal
- **Goal:** Congestion control without inducing loss
- Avoid forcing packet loss as a congestion signal. Instead, watch the delay/RTT: If RTT increases, queue is building → congestion coming. Reduce sending rate before the queue becomes full.
- Keep enough packets in-flight so the link stays fully utilized. No idle time, no wasted bandwidth → maximum throughput. but not fuller”Don’t overfill the queues.
- . TCP Vega
- 2. TCP fast
- 3. BBR (Google TCP) are examples

Explicit congestion notification (ECN)

- Traditional TCP detects congestion using packet loss. But ECN (Explicit Congestion Notification) allows the network itself to warn the sender before dropping packets. This is called network-assisted congestion control.
- ECN allows routers to mark packets instead of dropping them when congestion occurs. The CE mark in the IP header is sent to the receiver, which sets the TCP ECE bit in its ACKs to signal the sender. The sender then reduces cwnd and uses the TCP CWR bit to confirm the reaction. This method combines IP-level marking with TCP-level signaling.



Difference Between TCP and UDP

TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Connection-oriented protocol	Connectionless protocol
Reliable; guarantees delivery	Unreliable; no delivery guarantee
Ensures data arrives in order	No guarantee of order
Slower due to overhead	Faster and lightweight
Performs error control with retransmission	Error detection only; no retransmission
Has flow control and congestion control	No flow or congestion control
Larger header size (20 bytes)	Smaller header size (8 bytes)
Used for web, email, file transfer (HTTP, FTP, SMTP)	Used for online games, VoIP, streaming, DNS